One aspect you might find confusing from listing 20.3 is that for the successful case, we return a `string`, but the method signature of `View` says we return an `ActionResult<string>`. How is that possible? Why isn't there a compiler error?

The generic `ActionResult<T>` uses some fancy C# gymnastics with implicit conversions to make this possible. Using `ActionResult<T>` has two benefits:

- You can return either an instance of `T` *or* an `ActionResult` implementation like `NotFoundResult` from the same method. This can be convenient, as in listing 20.3.
- It enables better integration with ASP.NET Core's OpenAPI support.

You're free to return any type of `ActionResult` from your web API controllers, but you'll commonly return `StatusCodeResult` instances, which set the response to a specific status code, with or without associated data. `NotFoundResult` and `OkResult` both derive from `StatusCodeResult`, for example. Another commonly used status code is `400 Bad Request`, which is normally returned when the data provided in the request fails validation. You can generate this using a `BadRequestResult`, but in many cases the `[ApiController]` attribute can automatically generate `400` responses for you, as you'll see in section 20.5.

TIP You learned about various `ActionResult`s in chapter 15. `BadRequestResult`, `OkResult`, and `NotFoundResult` all